

MidEng GK8.3 CI/CD Pipelines in Jenkins [GK]

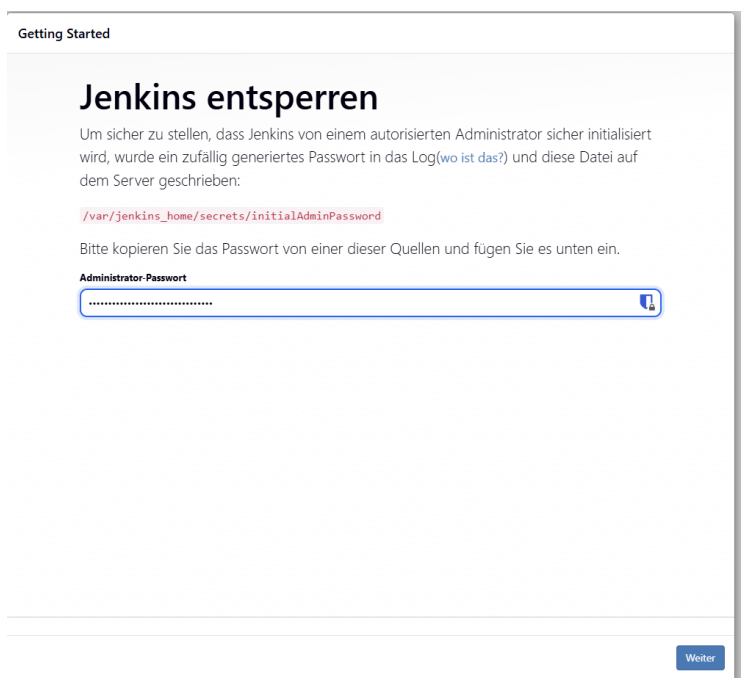
@author: David Socaciu & Maksym Kovacevic

@version: 2026-05-12

Installation & Vorbereitung

Als erstes haben wir uns den Jenkins Container in Docker gepullt und ausgeführt. Nachdem wir dies getan hatten und localhost:8080 verwendet haben wurden wir nach einem Admin-Passwort gefragt, welches wir hier auslesen konnten:

```
/var/jenkins_home/secrets/initialAdminPassword .
```



Anschließend haben wir uns die benötigten Plugins heruntergeladen.

Q Search available plugins				Install	Install		Released	Health
Install	Name	1						
☒	Docker	1316.v75635a_002b_0a_	Cloud-Agenten Cluster-Management und verteiltes Bauen docker				2 Monate 12 Tage ago	100
Provisions Jenkins agents on demand as Docker containers. Runs builds in isolated, disposable environments.								
☒	CloudBees Docker Build and Publish	1.4.0	Build-Werkzeuge docker				3 Jahre 8 Monate ago	89
This plugin enables building Dockerfile based projects, as well as publishing of the built images/repos to the docker registry.								

Danach haben wir über cmd getestet ob Docker richtig installiert wurde, und wir sehen, dass es auch so ist.

```
C:\Users\Socac>docker exec -it jenkins bash
root@808391f0712f:/# docker --version
^[[H Docker version 29.4.1, build 055a478
root@808391f0712f:/#
```

Jenkins Pipeline

Jetzt erstellen wir eine Pipeline.

Element anlegen

Geben Sie einen Element-Namen an

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.



"Free Style"-Softwareprojekt bauen

Der klassische, generische Projekt-Typ in Jenkins. Ein Build kann ein SCM auschecken, verschiedene Buildschritte nacheinander ausführen und anschließend Post-Build-Aktionen, wie E-Mailversand oder Archivierung von Artefakten, durchführen.



Multikonfigurationsprojekt bauen

Dieses Profil eignet sich sehr gut für Projekte mit zahlreichen Konfigurationen, die etwa in unterschiedlichen Umgebungen getestet oder plattformspezifisch gebaut werden müssen.



Multibranch Pipeline

Creates a set of Pipeline projects according to detected branches in one SCM repository.



Ordner

Erstellt einen Container, in dem verschachtelte Elemente gespeichert werden. Nützlich, um Dinge zu gruppieren. Im Gegensatz zur Ansicht, die nur ein Filter ist, erstellt ein Ordner einen separaten Namensraum, sodass Sie mehrere Dinge mit demselben Namen haben können, solange sie sich in verschiedenen Ordnern befinden.



Organization Folder

Creates a set of multibranch project subfolders by scanning for repositories.



Kopieren von

OK

Dann haben wir das Github Repo eingebunden und die Pipeline gebuildet.

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/ddsocaci/DEZSYS_JENKINS_HELLOSPENCER

Credentials ?

- leer -

+ Add

Erweitert

+ Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

main

+ Add Branch

Man sieht auch, dass sie korrekt funktioniert und in der Konsole, dass es erfolgreich war.



Jenkins / erste-pipeline

Status

</> Changes

▶ Jetzt bauen

⚙ Konfigurieren

🗑 Pipeline löschen

📁 Stages

✎ Rename

🔍 Pipeline Syntax

✓ erste-pipeline

Permalinks

- [Letzter Build \(#1\), vor 1 Minute 40 Sekunden](#)
- [Letzter stabiler Build \(#1\), vor 1 Minute 40 Sekunden](#)
- [Letzter erfolgreicher Build \(#1\), vor 1 Minute 40 Sekunden](#)
- [Neuester abgeschlossener Build \(#1\), vor 1 Minute 40 Sekunden](#)

Builds >

🔍 suchen

Today

✓ #1 07:24

```
[Pipeline] }
$ docker stop --time=1 114f01ed8f7422bdd7b902f8613ab8bd5d6d731d045f0d3e67d05a12f1306914
$ docker rm -f --volumes 114f01ed8f7422bdd7b902f8613ab8bd5d6d731d045f0d3e67d05a12f1306914
[Pipeline] // withDockerContainer
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Was uns noch fehlt ist, dass die Pipeline von einem Commit auf Github gestartet werden kann. Dafür mussten wir einen Trigger erstellen.

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☐ Starte Build, nachdem andere Projekte gebaut wurden ?
- ☐ Builds zeitgesteuert starten ?
- ☒ GitHub hook trigger for GITScm polling ?
- ☒ Source Code Management System abfragen ?

Zeitplan ?

Kein Zeitplan, deshalb kann dieses Projekt durch SCM-Änderungen nur mittels Post-Commit-Benachrichtigungen gestartet werden.

- ☐ Post-Commit-Benachrichtigungen ignorieren ?
- ☐ Builds von außerhalb starten (z.B. skriptgesteuert) ?

Und dann habe ich eine kleine Änderung im Code vorgenommen und comittet und man sieht, dass automatisch ein neuer Build erstellt wurde.



Jenkins / erste-pipeline ▾ / #2

Status

</> Changes

Console Output

✓ Edit Build Information

Abfrage-Protokoll

⌚ Timings

🔗 Git Build Data

🔗 Pipeline Overview

📄 Thread Dump

⏸ Pause/resume

↺ Replay

⋮ Pipeline Steps

📁 Workspaces

← Previous Build

✓ #2 (12.05.2026, 08:52:26)

Fortschritt:



Build wurde durch eine SCM-Änderung ausgelöst.



Dieser Durchlauf verbrachte 8.4 Sekunden wartend in der Warteschlange.



Revision: ed355b1cc9337121bc7c44ad5158838d806960cf

Repository: https://github.com/ddsocaci/DEZSYS_JENKINS_HELLOSPENCER

• origin/main



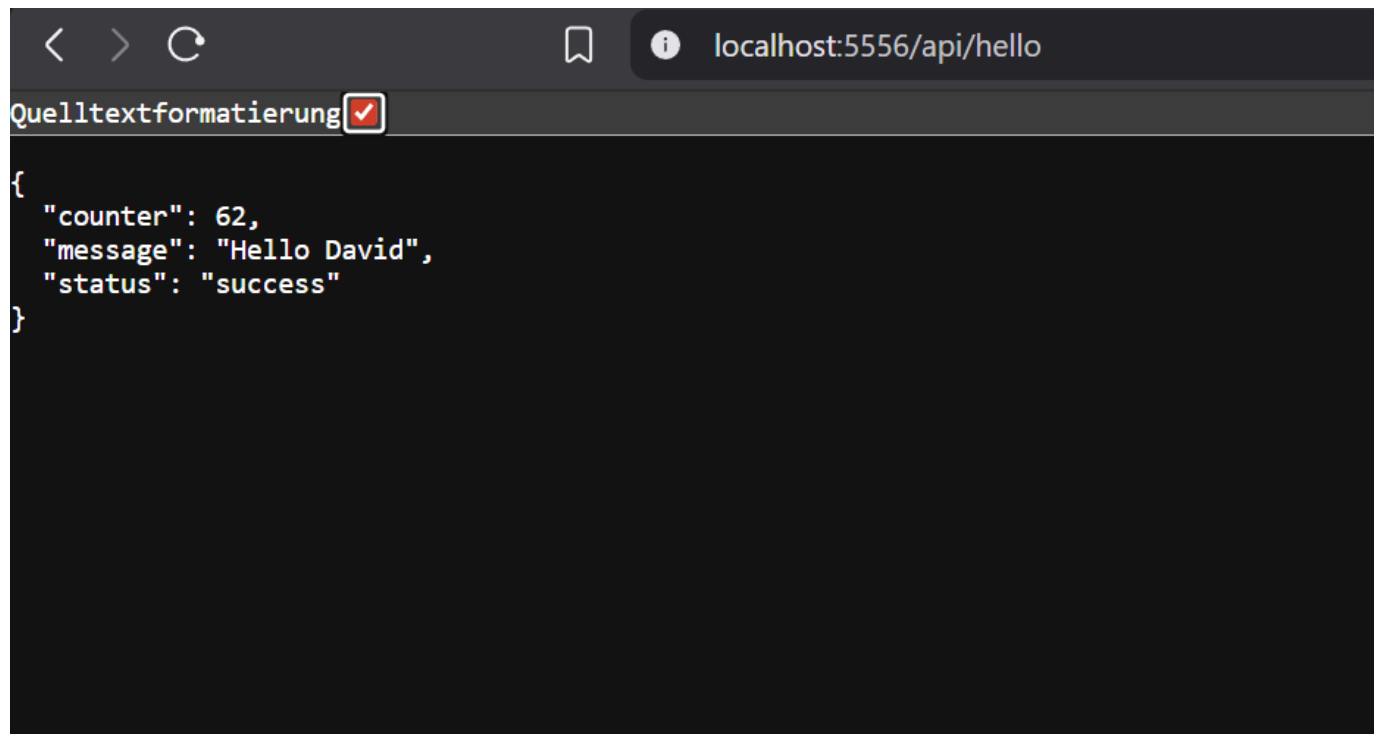
Test commit ed355b1

dsocaci at 08:52 12.05.26

Man sieht, dass wenn ich jetzt außerdem die Pipeline builde, dass ich auf die API zugreifen kann.

```
< > ↺ localhost:5556/api/hello
Quelltextformatierung ☐
{
  "counter": 34,
  "message": "Hello Spencer",
  "status": "success"
}
```

Dann haben wir den Namen von Spencer auf einen von uns umbenannt und nach dem Commit wurde das passend auch beim API-Aufruf geändert.



The screenshot shows a web browser window with the address bar displaying `localhost:5556/api/hello`. Below the address bar, there is a dark-themed editor area. At the top left of this area, the text `Quelltextformatierung` is visible next to a small red square icon containing a white checkmark. The editor displays a JSON object:

```
{  
  "counter": 62,  
  "message": "Hello David",  
  "status": "success"  
}
```